

Algoritmusok és adatszerkezetek I.

Rendezés lineáris időben

Eddigi ismereteink:
Összehasonlításon alapuló rendezések

Összehasonlításon alapuló rendezések korlátai

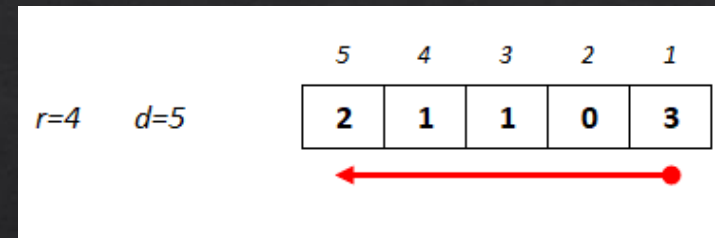
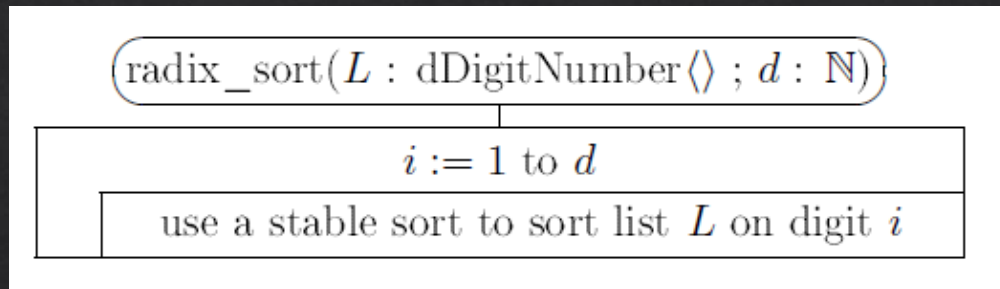
- ◊ 10.1. Tétel. Tetszőleges rendező algoritmusra $mT(n) \in \Omega(n)$
- ◊ 10.3. Tétel. Bármely összehasonlító rendezés végrehajtásához a legrosszabb esetben $MC(n) \in \Omega(n \lg n)$ kulcsösszehasonlítás szükséges.
- ◊ 10.4. Tétel. Tetszőleges összehasonlító rendezésre $MT(n) \in \Omega(n \lg n)$.
- ◊ Vegyük észre, hogy a heap sort és a merge sort, aszimptotikusan optimálisak abban az értelemben, hogy a műveletigényük $O(n \lg n)$ felsőkorlátja összeillik a (legrosszabb esetre vonatkozó) $MT(n) \in \Omega(n \lg n)$ alsó korláttal. Az előbbi tulajdonságokból azonnal következik, hogy mindkettőre $MT(n) \in \Theta(n \lg n)$.

Radix rendezés

(számjegypozíciós rendezés)

Radix rendezés elve

- ❖ Kihasználják, hogy a kulcsok r alapú számrendszerben felírt d számjegyű, előjel nélküli egész számok.
- ❖ Például $r=4$, $d=5$ esetén 21103 lehet egy kulcs.
- ❖ Az algoritmus:



- ❖ L egy absztrakt lista típus, dDigitNumber a fent definiált kulcsnak a típusa.

- ❖ Mitől lesz ez lineáris?

Az i -dik számjegy szerinti rendezés nem lehet a tanult összehasonlításon alapuló rendezési módszer, lineáris idejű algoritmusra van szükség.

- ❖ Miért fontos, hogy ez stabil rendezés legyen?

Az i -dik pozíció szerinti rendezésnél azonos számjegyek esetén meg kell őrizni az $i-1 \dots 1$ pozíciók szerinti rendezettségét.

I. - Szétválogató rendezés listákra (distribution sort)

A distribution sort alkalmazása radix rendezőként

- ♦ Fontos, hogy a bemenet egy lista! (Lehet egy vagy kétirányú, a tanult lista típusok valamelyike.)
- ♦ Helyiérték szerint visszafelé haladunk a számjegyeken: a legkevésbé szignifikáns helyiértékről indulva.
- ♦ r alapú számrendszer esetén $0..r-1$ listába szétfűzzük a kulcsokat az i -dik számjegy szerinti segéd listába, ügyelve, hogy a segéd lista végére kerüljön mindig az új elem. (Ehhez, ha szükséges, a lista elejét és végét is nyilván kell tartasuk egy-egy pointerrel.)
- ♦ Majd az így kapott listákat egymás után fűzzük.
- ♦ Mutassuk be a módszert a következő listán ($r=4$, $d=3$):

$L = \langle 101, 013, 310, 323, 003, 220, 121, 001, 302, 113 \rangle$

◇ Mutassuk be a módszert a következő listán ($r=4$, $d=3$):

$L = \langle 101, 013, 310, 323, 003, 220, 121, 001, 302, 113 \rangle$

L	101	013	310	323	003	220	121	001	302	113
0	310	220								
1	101	121	001							
2	302									
3	013	323	003	113						

1. menet:
a 3. számjegy szerint osztjuk
listákba az elemeket.

L	310	220	101	121	001	302	013	323	003	113
---	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----

$L = \langle 310, 220, 100, 121, 001, 302, 013, 323, 003, 113 \rangle$

Utolsó számjegy szerint
rendezett lett a lista.

$L = \langle 310, 220, 101, 121, 001, 302, 013, 323, 003, 113 \rangle$

L	310	220	101	121	001	302	013	323	003	113
0	101	001	302	003						
1	310	013	113							
2	220	121	323							
3										

2. menet:
a középső számjegy szerint
osztjuk listákba az elemeket.

L 101 001 302 003 310 013 113 220 121 323

$L = \langle 101, 001, 302, 003, 310, 013, 113, 220, 121, 323 \rangle$

Utolsó két számjegy
szerint rendezett lett a
lista.

$L = \langle 101, 001, 302, 003, 310, 013, 113, 220, 121, 323 \rangle$

L	101	001	302	003	310	013	113	220	121	323
0	001	003	013							
1	101	113	121							
2	220									
3	302	310	323							

3. (utolsó) menet:
a legnagyobb helyiértékű
számjegy szerint osztjuk listákba
az elemeket.

L 001 003 013 101 113 121 220 302 310 323

$L = \langle 001, 003, 013, 101, 113, 121, 220, 302, 310, 323 \rangle$

Kész a rendezés.

Szétválogató rendezés listákra (distribution sort)

- ◊ Adott az L lista.
- ◊ L elemeit szétosztjuk r segéd listába stabil módon (a segéd listák végére fűzve az elemeket).
- ◊ Majd a segéd listákat egymás után fűzve újra egyesítjük a lista darabokat. Itt érdemes visszafelé haladni, hogy az eredmény listának mindig az elejére fűzzük a következő lista darabot.

$\text{distributing_sort}(L : \mathcal{T}\langle \rangle ; r : \mathbb{N} ; \varphi : \mathcal{T} \rightarrow [0..r))$

$B : \mathcal{T}\langle \rangle[r] //$ array of lists, i.e. bins

$k := 0$ to $r-1$

Let $B[k]$ be empty list // init the array of bins

L is not empty

Remove the first element x of list L

Insert x at the end of $B[\varphi(x)] //$ stable distribution

$k := r-1$ downto 0

$L := B[k] + L //$ append $B[k]$ before L

Hatékonyság:
 $\Theta(n+r) = \Theta(n)$
 $(r \in O(n) \text{ miatt})$

A jegyzetben megtalálhatjuk C2L listákra az algoritmust

$\text{radix_sort}(L : E2^* ; d, r : \mathbb{N})$
$BinHead : E2[r]$ // the headers of the lists representing the bins
$B : E2^*[r]$ // pointers to the headers
$i := 0$ to $r - 1$
$B[i] := \&BinHead[i]$ // Initialize the i th pointer.
$i := 1$ to d
$\text{distribute}(L, i, B)$ // Distribute L on the i th digits of keys.
$\text{gather}(B, L)$ // Gather form the bins back into L

$\text{distribute}(L : E2^* ; i : \mathbb{N} ; B : E2^*[r])$
$L \rightarrow next \neq L$
$p := L \rightarrow next ; \text{out}(p)$
$\text{precede}(p, B[\text{digit}(i, r, p \rightarrow key)])$

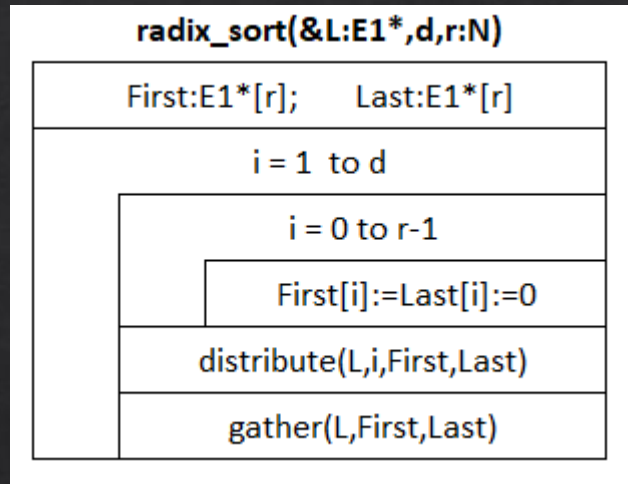
$\text{gather}(B : E2^*[r] ; L : E2^*)$
$i := 0$ to $r - 1$
$\text{append}(L, B[i])$ // add to the end of L the elements form $B[i]$

$\text{append}(L, Bi : E2^*)$
$Bi \rightarrow next \neq Bi$
$p := L \rightarrow prev ; q := Bi \rightarrow next ; r := Bi \rightarrow prev$
$p \rightarrow next := q ; q \rightarrow prev := p$
$r \rightarrow next := L ; L \rightarrow prev := r$
$Bi \rightarrow next := Bi ; Bi \rightarrow prev := Bi$
SKIP

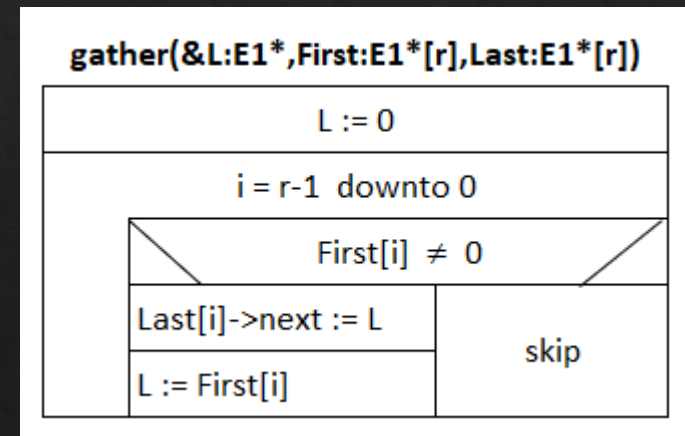
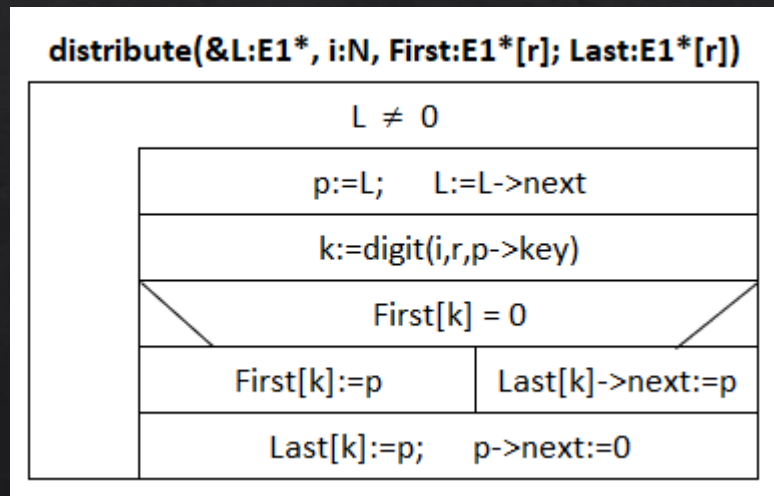
Lineáris a műveletigény?

$T_{\text{radix_sort}}(n, d, r) \in \Theta(r + d(n+r)) \rightarrow T_{\text{radix_sort}} \in \Theta(n)$ mivel d konstans, $r \in O(n)$

Készítsük el hasonlóan, S1L listára



A szétoztáshoz használt listákhoz két pointert fogunk használni, *First* az első elemre, *Last* az utolsó elemre mutat. Mivel r darab segéd listára lesz szükség *First* és *Last* r hosszúságú pointer tömbök.



II. - Leszámláló rendezés (counting sort)

A) - egy számjegyes esetben

Leszámláló rendezés (counting sort)

- ◊ Bemenet: A tömb
- ◊ Eredmény: B tömb
- ◊ Kulcsok a $0..r-1$ intervallumból származó egészek,
- ◊ Vagy leképezhetők egy φ függvénnyel a $0..r-1$ intervallumba.
- ◊ Műveletigény:
 $\Theta(n+r)$
 $r \in O(n)$ miatt: $T(n) \in \Theta(n)$

$\text{counting_sort}(A, B : \mathcal{T}[n] ; r : \mathbb{N} ; \varphi : \mathcal{T} \rightarrow [0..r))$	
$C : \mathbb{N}[r] \text{ // counter array}$	
$k := 0 \text{ to } r-1$	
	$C[k] := 0 \text{ // init the counter array}$
$i := 0 \text{ to } n-1$	
	$C[\varphi(A[i])]++ \text{ // count the items with the given key}$
$k := 1 \text{ to } r-1$	
	$C[k] += C[k-1] \text{ // } C[k] := \text{the number of items with key } \leq k$
$i := n-1 \text{ downto } 0$	
	$k := \varphi(A[i]) \text{ // } k := \text{the key of } A[i]$
	$C[k] -- \text{ // The next one with key } k \text{ must be put before } A[i] \text{ where}$
	$B[C[k]] := A[i] \text{ // Let } A[i] \text{ be the last of the \{unprocessed items with key } k\}}$

- ◇ Első lépés:
- ◇ a kulcsok (vagy leképezett értékük) leszámolása $C[0..r-1]$ tömbben: kulcsok gyakoriságát kapjuk.
- ◇ A gyakoriság kumulálása.

$\text{counting_sort}(A, B : \mathcal{T}[n] ; r : \mathbb{N} ; \varphi : \mathcal{T} \rightarrow [0..r))$

$C : \mathbb{N}[r]$ // counter array

$k := 0$ to $r-1$

$C[k] := 0$ // init the counter array

$i := 0$ to $n-1$

$C[\varphi(A[i])]++$ // count the items with the given key

$k := 1$ to $r-1$

$C[k] += C[k-1]$ // $C[k] :=$ the number of items with key $\leq k$

$i := n-1$ downto 0

$k := \varphi(A[i])$ // $k :=$ the key of $A[i]$

$C[k]--$ // The next one with key k must be put before $A[i]$ where

$B[C[k]] := A[i]$ // Let $A[i]$ be the last of the {unprocessed items with key k }

	1	2	3	4	5	6	7	8
A=	3	0	2	5	1	5	0	2

	0	1	2	3	4	5
C=	2	1	2	1	0	2

kulcsok gyakorisága: $C[i] =$ hányszor fordul elő az i kulcs A-ban

	0	1	2	3	4	5
C=	2	3	5	6	6	8

kumulált gyakoriság: $C[i] = 0..i$ kulcsok együttes gyakorisága

Második lépés: kulcsok helyre mozgatása

$\text{counting_sort}(A, B : \mathcal{T}[n] ; r : \mathbb{N} ; \varphi : \mathcal{T} \rightarrow [0..r))$

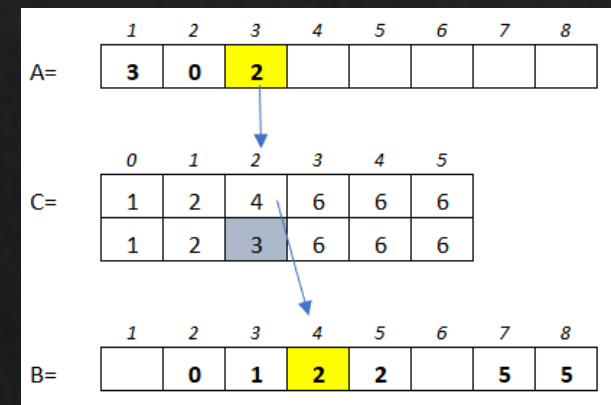
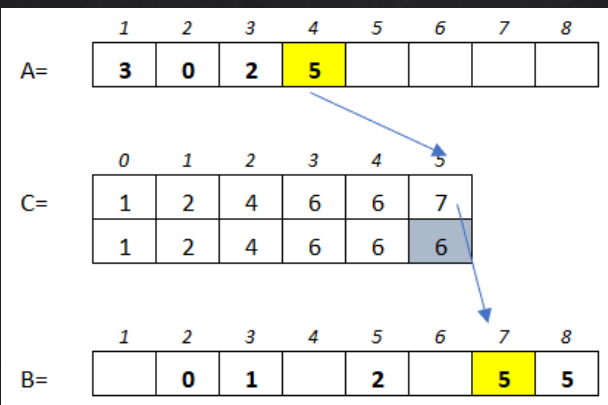
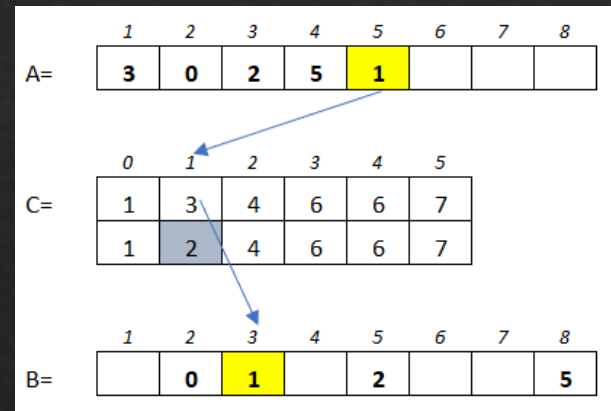
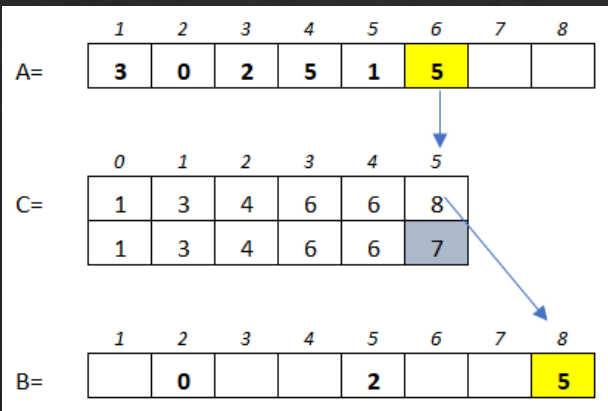
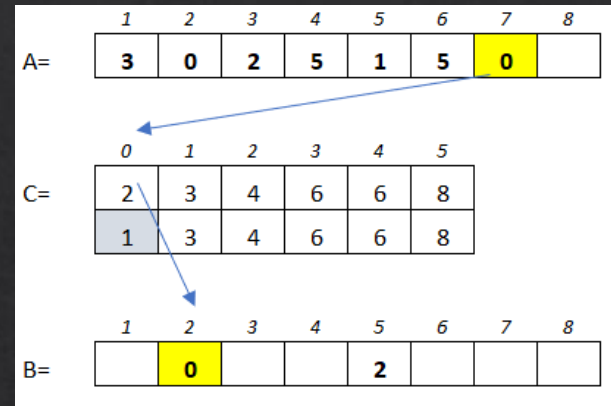
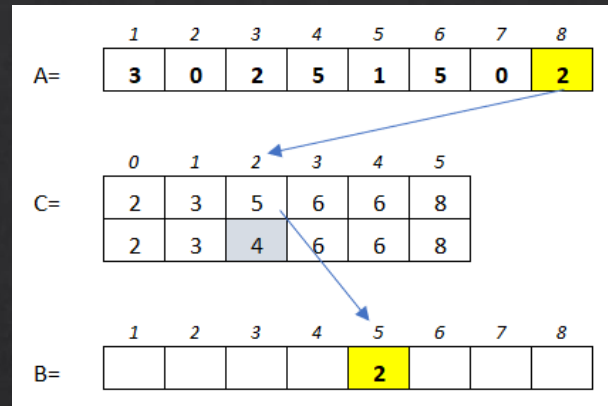
$C : \mathbb{N}[r]$ // counter array
$k := 0$ to $r-1$
$C[k] := 0$ // init the counter array
$i := 0$ to $n-1$
$C[\varphi(A[i])]++$ // count the items with the given key
$k := 1$ to $r-1$
$C[k] += C[k-1]$ // $C[k] :=$ the number of items with key $\leq k$
$i := n-1$ downto 0
$k := \varphi(A[i])$ // $k :=$ the key of $A[i]$
$C[k]--$ // The next one with key k must be put before $A[i]$ where
$B[C[k]] := A[i]$ // Let $A[i]$ be the last of the $\{\text{unprocessed items with key } k\}$

A-ból a kulcsokat áthelyezzük B-be a rendezettség szerinti helyükre, a C tömb segítségével.

Ha A-n visszafelé haladunk, stabil rendezést kapunk.

Miért?

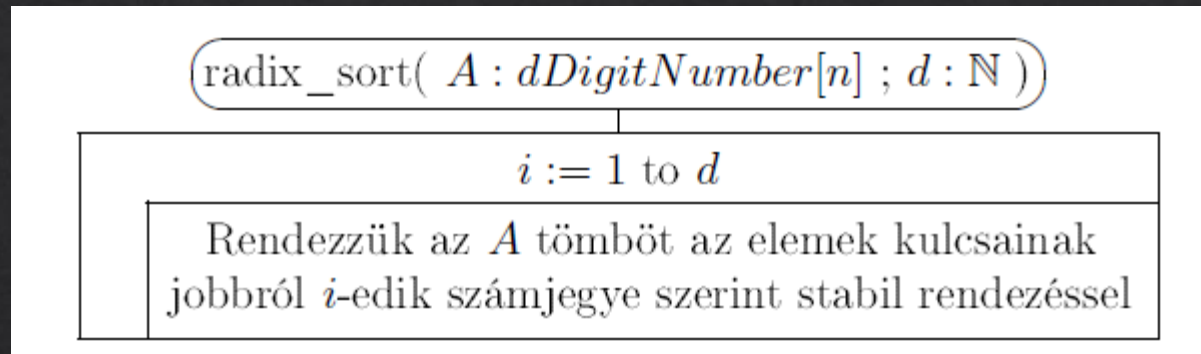
Megjegyzés: az ábrán A és B 1-től indexelt tömbök.



II. - Leszámláló rendezés (counting sort)

B) - több számjegyes esetben

Radix rendezés tömbben tárolt kulcsok esetén, belül leszámpláló rendezést használva



- ◆ Legyenek most a kulcsok egy tömbben (r = számrendszer, d = számjegyek száma)
- ◆ A helyiértékeken most is jobbról balra haladunk, de az i -edik számjegy szerinti rendezéshez leszámpláló rendezést alkalmazunk.
- ◆ Felváltva két tömböt használunk, egyikből a másikba mozgatjuk a kulcsokat (rekordokat).
- ◆ Szemléltessük a módszert a következő tömbön ($r=4$, $d=2$):
 $\langle 11, 20, 10, 23, 21, 30 \rangle$

- ◊ Szemléltessük a módszert a következő tömbön ($r=4$, $d=2$):
 $\langle 11, 20, 10, 23, 21, 30 \rangle$

$A = \langle 11, 20, 10, 23, 21, 30 \rangle$

Leszámlálás a második számjegy szerint:

		11	20	10	23	21	30			30	21	23	10	20	11
0	0		1	2			3	3	3	2			1	0	
1	0	1				2		2	5		4				3
2	0							0	5						
3	0				1			1	6			5			

Helyre rakáskor a C tömbből olvassuk ki, hova kell tenni az elemet, majd csökkentjük eggyel C megfelelő értékét.

B=	1	2	3	4	5	6
	20	10	30	11	21	23

$B = \langle 20, 10, 30, 11, 21, 23 \rangle$

$B = \langle 20, 10, 30, 11, 21, 23 \rangle$

Leszámlálás az első számjegy szerint:

		20	10	30	11	21	23			23	21	11	30	10	20
0	0							0	0						
1	0		1		2			2	2			1		0	
2	0	1				2	3	3	5	4	3				2
3	0			1				1	6				5		

A=	1	2	3	4	5	6
	10	11	20	21	23	30

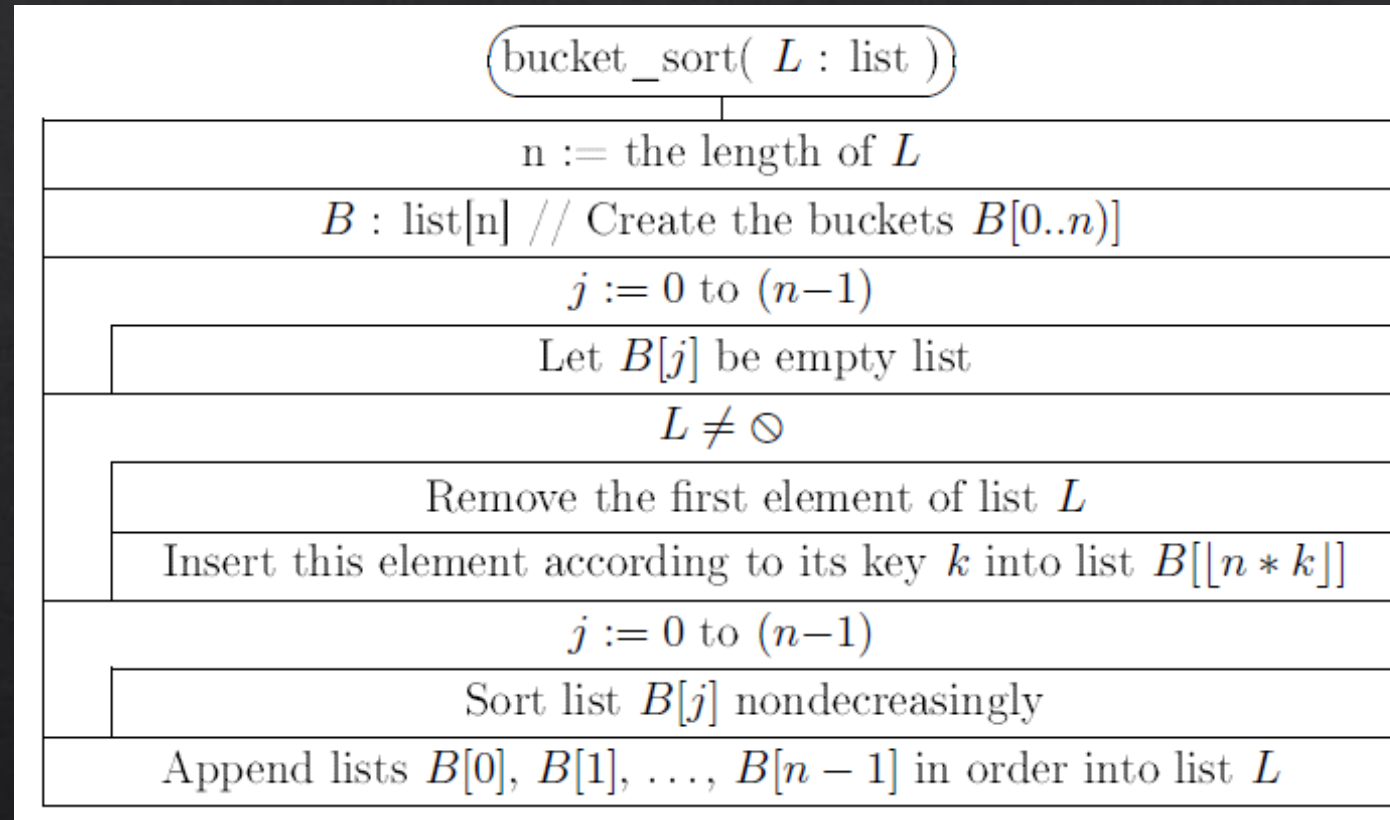
$A = \langle 10, 11, 20, 21, 23, 30 \rangle$

Páros darabszámú számjegy esetén a rendezendő számok épp abban a tömbben vannak, amelyikben eredetileg voltak.

III. - Edényrendezés (bucket sort) A) – „Egyszerű”

Edényrendezés (bucket sort)

- ♦ Feltesszük, hogy a rendezendő elemek kulcsai a $[0; 1)$ valós intervallum elemei.
- ♦ Ha az adatok száma n , akkor n darab edényt készítünk: $B[0..n-1]$
- ♦ Képezzük az $\lfloor n * k \rfloor$ edénysorszámot, és betesszük a kulcsot az adott edénybe.
- ♦ Lerendezzük az edények tartalmát, majd összefűzzük az edényeket.
- ♦ Nyilván $mT(n) \in \Theta(n)$, és ha a kulcsok eloszlása egyenletes, akkor $AT(n) \in \Theta(n)$ is teljesül.
- ♦ $MT(n)$ pedig attól függ, hogy a $B[j]$ listákat milyen módszerrel rendezzük. Pl. egyszerű beszűrő rendezést használva $MT(n) \in \Theta(n^2)$, összefésülő rendezéssel viszont $MT(n) \in \Theta(n \lg n)$.



Bucket sort szemléltetése

Az algoritmus lejátszása a $\langle 0.79, 0.13, 0.16, 0.64, 0.39, 0.20, 0.89, 0.53, 0.71, 0.42 \rangle$ listán:

	Beszúrás után		Rendezés után	Amelyik listába több kulcs is kerül, ott mindig a lista elejére szúrjuk be az új elemet, majd lerendezzük a listákat.
0	$\diamond$		$\diamond$	
1	$\langle 0.13 \rangle$	$\langle 0.16, 0.13 \rangle$	$\langle 0.13, 0.16 \rangle$	
2	$\langle 0.20 \rangle$		$\langle 0.20 \rangle$	
3	$\langle 0.39 \rangle$		$\langle 0.39 \rangle$	
4	$\langle 0.42 \rangle$		$\langle 0.42 \rangle$	
5	$\langle 0.53 \rangle$		$\langle 0.53 \rangle$	
6	$\langle 0.64 \rangle$		$\langle 0.64 \rangle$	
7	$\langle 0.79 \rangle$	$\langle 0.71, 0.79 \rangle$	$\langle 0.71, 0.79 \rangle$	
8	$\langle 0.89 \rangle$		$\langle 0.89 \rangle$	
9				

Végeredmény: $\langle 0.13, 0.16, 0.20, 0.39, 0.42, 0.53, 0.64, 0.71, 0.79, 0.89 \rangle$

Bucket sort algoritmus S1L listára

bucket_sort(&L : E1*)

n := length(L)

Z : E1*[n] //edények listáinak első elemére mutató pointerok tömbje

j = 0 to n-1

Z[j] := 0;

L ≠ 0

p, L := L, L->next

j := [n * p->key]

p->next := Z[j] Z[j] := p //lista elejére fűzünk

j = 0 to n-1

sort(Z[j])

appendLists(L,Z)

- ◇ Length(L:E1*):N ismert függvény, a jegyzetben megtalálható
- ◇ Rendezéshez használható a merge sort: merge(&L:E1*)
- ◇ appendLists(&L:E1*, Z:E1*[n]) összefűzi a listákat.

III. - Edényrendezés (bucket sort) B) – Bináris radix rendezők

Bináris radix rendezők

- ◊ A kettes számrendszerben történő ábrázolás miatt fontos szerephez jutottak a *bináris radix rendezők*.
- ◊ Két változatát nézzük meg:
 - ◊ (1) Két tömböt felváltva használó, „bináris radix vissza” rendező.
 - ◊ (2) Egy tömbben rendező „bináris radix előre” rendező.
- ◊ Az algoritmusok csak érdekességek, vizsgán nem szerep

(1) „Bináris radix vissza” (2 tömbös)

- ◆ **A helyiértékeket „visszafelé” a legkisebbtől a legnagyobb felé dolgozzuk fel.**
(Listás radixhoz hasonló, de tömbben dolgozik.)
- ◆ Amennyiben bináris számokat rendezünk, mivel csak kétfelé kell osztani a számokat, két „edény” kell a szétszétályozáshoz.
- ◆ Megtehetjük azt, hogy a nullások a tömb elejétől: az 1-es indextől növekvőleg, az egyesek pedig a tömb végétől csökkenőleg lesznek elhelyezve.
- ◆ Így haladunk a legkisebb helyiértéktől visszafelé, ügyelve arra, hogy elsőként mindig a nullás edényt dolgozzuk fel 1-től növekvőleg, majd az egyes edényt n-től csökkenőleg.
- ◆ Ez a feldolgozási irány biztosítja a stabilitást.

Szemléltetés

- ◊ Szemléltessük a rendezést az $A = \langle 101, 001, 011, 110, 010, 111, 000 \rangle$ tömbön.
- ◊ Az A tömb elemeit dolgozzuk fel.
- ◊ Ha 1-re végződik akkor az 1-es edénybe kerül. Az 1-es edény a B tömb „végén” kezdődik, és a B közepe felé növekszik.
(Kék nyíl.)
- ◊ Ha nullára végződik, akkor a 0-ás edénybe kerül. A nullás edény a B tömb elején kezdődik és a közepe felé nő.
(Sárga nyíl.)

A	B	B	B	B	B	B	B
101				110	110	110	110
001					010	010	010
011							000
110						111	111
010			011	011	011	011	011
111		001	001	001	001	001	001
000	101	101	101	101	101	101	101



Folytatás a második bit szerint

- ◇ B-ből A-ba szortírozunk.
- ◇ Elsőként a nullás edényt dolgozzuk fel, olvasása 1..n irányú.
- ◇ Utána az 1-es edényt, olvasása n..1 irányú.
- ◇ Az edények most az A tömbben keletkeznek, az előző menethez hasonlóan.

	B	A	A	A	A	A	A	A
1.	110			000	000	000	000	000
	010				101	101	101	101
	000					001	001	001
2	111							111
	011						011	011
	001		010	010	010	010	010	010
	101	110	110	110	110	110	110	110

Folytatás a harmadik
(legnagyobb helyiértékű)
bit szerint

- ◇ A-ból B-be szortírozunk.
- ◇ Elsőként a nullás edényt dolgozzuk fel, olvasása 1..n irányú.
- ◇ Utána az 1-es edényt, olvasása n..1 irányú.
- ◇ Az edények most újra a B tömbben keletkeznek, az előző menethez hasonlóan.

	A	B	B	B	B	B	B	B
1	000	000	000	000	000	000	000	000
	101			001	001	001	001	001
	001					010	010	010
2	111						011	011
	011							111
	010				110	110	110	110
	110		101	101	101	101	101	101

Utolsó menet

- ◇ Eltér az eddigi menetektől.
- ◇ B-ből A-ba szortírozunk.
- ◇ A nullás edényt átmásoljuk A-ba.
- ◇ Az 1-es edény tartalmát megfordítjuk: n..1 irányból olvasva, átpakoljuk A-ba a nullás edény mögé.
- ◇ Ezzel előállt a rendezett tömb.

	B	A	A	A	A	A	A	A
1	000	000	000	000	000	000	000	000
	001		001	001	001	001	001	001
	010			010	010	010	010	010
	011				011	011	011	011
2	111					101	101	101
	110						110	110
	101							111

(2) - „Bináris radix előre” rendező (1 tömbös)

- ◊ **Bináris radix előre rendező egy tömbbel, a helyiértékeket a legnagyobbtól a legkisebb felé dolgozza fel, egy meneten belül cserélgeti az elemeket.**
- ◊ Nem negatív bináris számokat úgy is rendezhetünk radix módszerrel, hogy a legnagyobb helyiértékekkel kezdünk, cserélgetésekkel átalakítjuk úgy, hogy a 0 kezdő bitesek elől legyenek, az 1 kezdő bitesek pedig a tömb második felében.
- ◊ Ehhez a tömb elejétől indulva keresünk egy 1-es bittel kezdődő számot, majd a tömb végéről indulva keresünk egy 0-s bittel kezdődőt, ha a két kapott index különbözik, felcseréljük őket.
- ◊ Folytatjuk, amíg össze nem ér a két index. Ekkor a tömb elején már csupa 0 kezdő bites szám lesz, a tömb végén pedig az 1-es bittel kezdődők.
- ◊ Rekurzívan folytatjuk ugyanezt a nullás és egyes edényre.

Szemléltetés

◊ Mutassuk be az algoritmus működését a $\langle 101, 001, 011, 110, 010, 111, 000 \rangle$ tömbön:

101	000	000	E0	000	E00	000	000	000
001	001	001		001		001	001	001
011	011	011		011	E01	011	011	010
110	110	010		010		010	010	011
010	010	110	E1	110	E10	101	101	101
111	111	111		111	E11	111	111	110
000	101	101		101		110	110	111
<i>csere</i>	<i>csere</i>			<i>csere</i>			<i>csere</i>	<i>kész</i>

Az algoritmus struktogramja

- ◇ A rendezendő tömrész: $ah..fh$
- ◇ Az i -dik bit szerint rendezünk.
- ◇ $i0$ index indul elsőként, ah -ról, és megáll, ha egy 1-es bittel találkozik.
- ◇ Elindul $i1$ is, csökkenőleg lépeget, és megáll, ha egy nullás bittel találkozik.
- ◇ Ha nem ért össze a két index, csere következik, majd $i0$ és $i1$ újra elindulnak a cseréhez képest eggyel eltolt pozíciótól.

